Nombre: Emmanuel Maldonado Martínez	**Matrícula**: 7182641
Nombre del curso: Gestión de Proyectos de Inteligencia Artificial	**Nombre del profesor**: Luis Ariel Vazquez Piña
Módulo: Implementación ágil y ciclo de vida de soluciones de IA	**Actividad 5.** Design thinking, definiendo e ideando
Fecha: 18 de octubre de 2026	
Bibliografía: En índice	

Actividad 5. Design thinking, definiendo e ideando

Instrucciones

Selecciona al menos dos algoritmos, por ejemplo, regresión logística frente a *random forest* para clasificación, o regresión lineal frente a *XGBoost* para regresión, con el fin de realizar el entrenamiento, el ajuste y el registro. Asimismo, deberás definir métricas estandarizadas, ejecutar validación cruzada (*k-fold* o *grid search*) y registrar los resultados en MLflow.

1. **Preparación del entorno y repositorio.**

1. Crea un repositorio en GitHub con la siguiente estructura mínima:

```
Actividad5/
    ├── datos/
    │   ├── datos_ini # datos originales
    │   └── datos_limp # datos limpios
    ├── fuentes/
    │   ├── entrena.ipynb # notebook de Colab (entrenamiento y evaluación)
    │   ├── datos_prep.py # funciones de limpieza
    │   └── train.py # script de entrenamiento y registro en MLflow
    ├── README.md # guía de reproducción y decisiones
    ├── CHANGELOG.md # reporte final
```

    2. Inicializa Git y conéctalo con GitHub.
2. **Configura MLflow** (seguimiento local o remoto) y valida que se registren parámetros, métricas y artefactos.
3. **Genera el repositorio en GitHub**.
    1. Dataset versionado (o *scripts* de obtención).
    2. Código en Python (*.py*) o *notebook* (*.ipynb*).
    3. Evidencias de MLflow.
    4. Archivo README.md documentado.
4. **Reporte técnico en PDF.**
    1. Debe contener:
        1. Descripción del dataset (documentar fuente, diccionario de datos y supuestos).
        2. Proceso de limpieza y estandarización (manejo de valores nulos, tipos de datos, escalado o normalización, codificación de variables categóricas y reglas de negocio).
        3. Estrategia de versionamiento (registrar cambios en CHANGELOG.md o en el README).
        4. Visualizaciones relevantes (identificación de patrones y anomalías).
        5. Descripción de los modelos entrenados.
        6. Configuración de hiperparámetros.
        7. Métricas obtenidas (clasificación: *accuracy*, *precision*, *recall*, *F1*, ROC-AUC; regresión: MAE, RMSE, R^2).
        8. Comparación estadística de resultados.
        9. Evidencia del uso de MLflow.
        10. Reflexión sobre deuda técnica y MLOps.
5. **Recopila evidencias visuales.**
    1. Capturas del panel de MLflow.
    2. Tablas o gráficas comparativas de desempeño.

Universidad Tecmilenio

Índice

0. Ligas de acceso	4
1. Descripción del dataset	4
2. Proceso de limpieza y estandarización	5
3. Estrategia de versionamiento	6
4. Visualizaciones relevantes	7
5. Descripción de modelos entrenados	8
6. Configuración de hiperparámetros	8
7. Métricas obtenidas	9
8. Comparación estadística de resultados	11
9. Evidencia de uso de ML Flow	12
10. Reflexión de deuda técnica y MLOps	15

0. Ligas de acceso

- Liga del entorno Colab: https://colab.research.google.com/drive/1592CRgrXocOlcT3LKx51P2kkek1RUQ6q#scrollTo=C2raP9cYFvd-
- Liga del repositorio: https://github.com/EmmaMMtz/Actividad-5

1. Descripción del dataset

Se analizan 45,211 observaciones de campañas de marketing bancario para predecir la contratación de un depósito a plazo, obtenido de un dataset público. Se hicieron las comparaciones usando *Regresión Logística* y *Random Forest* con búsqueda de hiperparámetros y validación cruzada de cinco particiones. Random Forest alcanzó ROC-AUC de 0.9281 frente a 0.9061 de Regresión Logística en el conjunto de pruebas. La diferencia es significativa sobre el resultado de ambos programas. Las evidencias locales de MLflow permiten ver los resultados de ambos programas.

Como se especifico antes, el dataset se extrajo de la siguiente fuente pública: UCI Machine Learning Repository, Bank Marketing (ID 222), https://archive.ics.uci.edu/dataset/222/bank+marketing.

Archivo examinado: bank_marketing_original.csv. Tamaño: 45,211 filas, 16 variables predictoras y una variable objetivo (y). La clase positiva representa la contratación del depósito (yes=1). A continuación se describen los tipos de datos presentados.

Variable(s)	**Tipo / significado**
age	Numérica: edad del cliente
job, marital, education	Categóricas: ocupación, estado civil, educación
default, housing, loan	Categóricas: impago, crédito hipotecario, préstamo personal
balance	Numérica: saldo de cuenta
contact, month	Categóricas: canal y mes del contacto

day_of_week	Numérica en este archivo: día del mes del contacto (nombre heredado)
duration	Numérica: duración del contacto en segundos
campaign, pdays, previous	Numéricas: contactos actuales, días desde contacto previo y contactos previos
poutcome	Categórica: resultado de campaña previa
y	Objetivo binario: yes/no en origen, 1/0 en versión preparada

Supuestos: se interpreta cada fila como un contacto/registro de campaña; los códigos especiales y valores faltantes requieren reglas explícitas. El atributo duration podría no estar disponible antes de realizar la llamada y debe evaluarse según el momento real de predicción.

2. Proceso de limpieza y estandarización

Se verificaron ambos CSV aportados. La versión original y la preparada tienen idéntica forma (45,211 × 17). La transformación confirmada entre archivos es la codificación de y: no→0 y yes→1. En ambas versiones permanecen 52,124 celdas faltantes distribuidas entre variables predictoras; por tanto, no es correcto declarar una imputación o eliminación completa de nulos.

Columna	Nulos	Porcentaje
poutcome	36,959	81.7 %
contact	13,020	28.8 %
education	1,857	4.1 %
job	288	0.6 %

Para realizar el entrenamiento de los modelos, primero fue necesario preparar la información del conjunto de datos Bank Marketing, ya que contiene variables numéricas y categóricas que requieren diferentes tratamientos.

En el caso de las variables numéricas, como la edad y el saldo bancario, se aplicó una estandarización para que sus valores se encontraran en escalas comparables. Por otro lado, las variables categóricas, como la ocupación y el

estado civil, se convirtieron en valores numéricos que los algoritmos pudieran interpretar.

Posteriormente, el conjunto de datos se dividió en dos partes: el 80 % se utilizó para entrenar los modelos y el 20 % restante para evaluar su desempeño. Esta división permitió comprobar cómo responden los algoritmos ante información que no habían utilizado durante el entrenamiento.

También se procuró mantener una proporción similar de clientes que contrataron y no contrataron el depósito bancario en ambos conjuntos.

Las transformaciones se organizaron dentro del proceso de entrenamiento y validación cruzada, evitando que los modelos utilizaran información de los datos de evaluación antes de tiempo. Finalmente, aunque se identificaron valores faltantes en algunas variables, no se estableció una regla específica para sustituirlos. Por ello, este aspecto se considera una oportunidad de mejora para futuras versiones del proyecto.

3. Estrategia de versionamiento

El código y los datos se organizaron en GitHub bajo datos/datos_ini, datos/datos_limp y fuentes; el notebook, los scripts y las evidencias se documentaron en README.md y CHANGELOG.md. Los CSV proporcionan instantáneas del origen y de la preparación; el historial de commits permite rastrear modificaciones. Se recomienda para futuros proyectos añadir requirements.txt con versiones fijadas, pruebas de calidad de datos y referencias a los identificadores de ejecuciones MLflow.

4. Visualizaciones relevantes

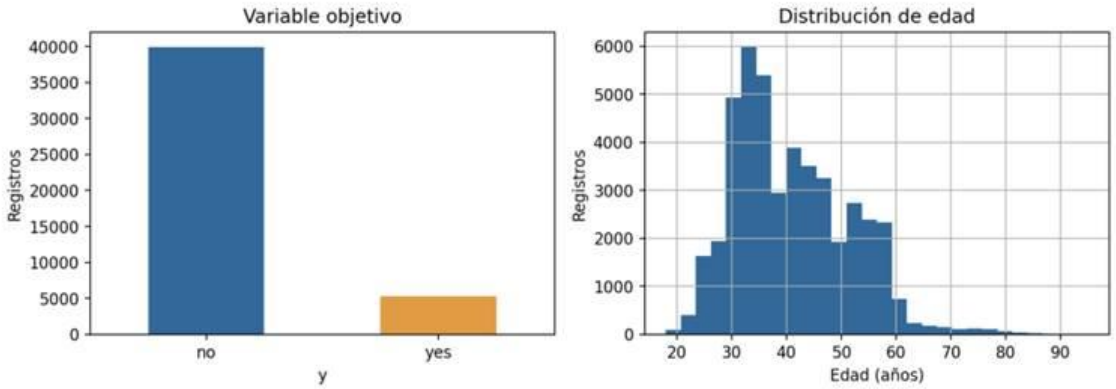


Figura 1. Distribución de la clase objetivo y de la edad. El desbalance de clases obliga a complementar accuracy con recall, F1 y ROC-AUC.

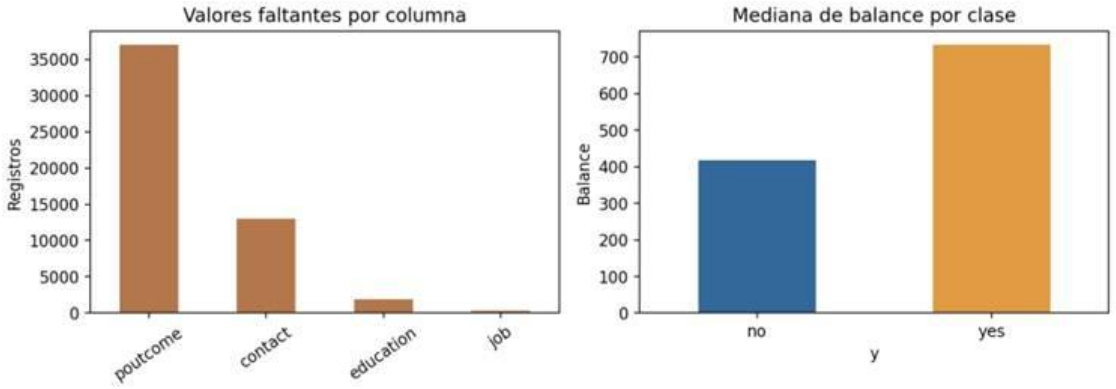


Figura 2. Conteo de faltantes y mediana de balance por clase. Los valores faltantes y el tratamiento de códigos especiales son aspectos de calidad de datos que requieren reglas explícitas.

Las gráficas dieron a conocer mejor los datos antes de entrenar los modelos. Primero identifiqué que la mayoría de los clientes no contrató el depósito, por lo que existe un desequilibrio entre las categorías. También observé que gran parte de los clientes tiene entre 30 y 40 años. Después se revisó qué variables tenían información faltante y encontré que el resultado de campañas anteriores

era una de las más afectadas. Finalmente, se observó que los clientes que sí contrataron el depósito tenían, en general, una mediana de saldo bancario mayor. Estos hallazgos permitieron comprender mejor los datos y reconocer qué aspectos debía considerar al evaluar los modelos.

5. Descripción de modelos entrenados

Para desarrollar el proyecto se seleccionaron dos algoritmos de clasificación: Regresión Logística y Random Forest. El objetivo fue comparar su desempeño para identificar qué modelo tenía una mayor capacidad para predecir si un cliente contrata un depósito bancario. A continuación se describe el motivo para usar cada uno:

- Regresión Logística: Se utilizó como modelo de referencia debido a que permite estimar la probabilidad de que un cliente contrate o no el producto bancario. Una de sus principales ventajas es que su funcionamiento es relativamente sencillo y facilita la interpretación de los resultados.
- Random Forest: Se seleccionó como segundo algoritmo porque combina varios árboles de decisión para generar predicciones. Esto le permite analizar diferentes características de los clientes e identificar relaciones más complejas entre las variables, como la edad, el saldo bancario y los antecedentes de campañas comerciales.

6. Configuración de hiperparámetros

Para mejorar el desempeño de ambos modelos se utilizó GridSearchCV, una herramienta que permite evaluar distintas configuraciones y seleccionar la que presenta mejores resultados. Se aplicó validación cruzada de cinco particiones y se utilizó ROC-AUC como métrica principal para seleccionar la mejor configuración.

Las configuraciones seleccionadas fueron las siguientes:

Modelo	Mejor configuración reportada
Regresión Logística	C=0.1; max_iter=1000
Random Forest	n_estimators=100; max_depth=20

Adicionalmente, se estableció una semilla aleatoria de 42 para facilitar la reproducción de los resultados y se reservó el 20 % del conjunto de datos para evaluar los modelos. Finalmente, los resultados del entrenamiento realizado en Google Colab se incorporaron posteriormente a MLflow local, donde se registraron las métricas y los parámetros utilizados. Esto permitió conservar un seguimiento organizado de los experimentos y facilitar la comparación entre los algoritmos.

7. Métricas obtenidas

Métrica	Regresión Logística	Random Forest	Diferencia RF–LR
Accuracy	0.9005	0.9062	0.0058
Precision	0.6411	0.6762	0.0351
Recall	0.3393	0.3809	0.0416
F1-score	0.4438	0.4873	0.0435
ROC-AUC	0.9061	0.9281	0.022
ROC-AUC CV (mejor)	0.907	0.9293	0.0224

Para evaluar el desempeño de los modelos de Regresión Logística y Random Forest, se utilizaron diferentes métricas de clasificación, con el propósito de analizar su capacidad para predecir si un cliente contrata un depósito bancario.

Los resultados muestran que ambos algoritmos alcanzaron una exactitud superior al 90 %. Sin embargo, Random Forest obtuvo un resultado ligeramente mayor, con 90.62 %, frente al 90.05 % de Regresión Logística. Esto indica que ambos modelos realizaron correctamente la mayoría de las clasificaciones, aunque Random Forest presentó una pequeña ventaja.

En cuanto a la precisión, Random Forest alcanzó 67.62 %, mientras que Regresión Logística obtuvo 64.11 %. Esto significa que Random Forest fue más preciso al identificar a los clientes que probablemente contraten el producto bancario.

Respecto al Recall, los resultados fueron de 38.09 % para Random Forest y 33.93 % para Regresión Logística. Aunque Random Forest obtuvo un mejor desempeño, ambos modelos presentan oportunidades de mejora, ya que no lograron identificar a una proporción importante de los clientes que realmente contrataron el depósito.

El F1-score también favoreció a Random Forest, con 48.73 %, frente al 44.38 % de Regresión Logística. Este resultado indica un mejor equilibrio entre precisión y capacidad de identificación de clientes positivos.

Por otra parte, la métrica ROC-AUC mostró valores de 0.9281 para Random Forest y 0.9061 para Regresión Logística, lo que demuestra una mayor capacidad de Random Forest para distinguir entre clientes que contrataron y no contrataron el depósito.

Finalmente, los resultados de validación cruzada fueron consistentes con esta comparación, ya que Random Forest obtuvo un ROC-AUC promedio de 0.9293, mientras que Regresión Logística alcanzó 0.9070.

En general, los resultados indican que Random Forest presentó un mejor desempeño en todas las métricas evaluadas. No obstante, los valores de Recall muestran que todavía existen oportunidades para mejorar la identificación de clientes que efectivamente contraten el producto bancario.

8. Comparación estadística de resultados

Con el propósito de comparar el desempeño de los modelos entrenados, se analizaron las diferencias entre las métricas obtenidas por Regresión Logística y Random Forest. Los resultados muestran que Random Forest presentó un mejor desempeño en las evaluaciones realizadas. En la métrica ROC-AUC obtuvo una ventaja aproximada de 2.20 puntos porcentuales respecto a Regresión Logística. Asimismo, presentó mejores resultados en F1-score y Accuracy, con diferencias aproximadas de 4.35 y 0.57 puntos porcentuales, respectivamente.

Al revisar los resultados de validación cruzada, se observó un comportamiento similar. Random Forest alcanzó un ROC-AUC promedio de 0.9293, mientras que Regresión Logística obtuvo 0.9070. Esto indica que Random Forest mantuvo un mejor desempeño promedio durante el proceso de evaluación.

Sin embargo, es importante señalar que estas diferencias corresponden a una comparación descriptiva. Aunque permiten identificar qué modelo obtuvo mejores resultados, no son suficientes para determinar si la diferencia es estadísticamente significativa. Para realizar una evaluación estadística más completa sería necesario contar con los resultados individuales de cada partición de validación cruzada y analizar la variabilidad de las métricas obtenidas.

Por esta razón, no se realizó una prueba de significancia estadística. En su lugar, se presenta una comparación cuantitativa de las métricas disponibles, reconociendo esta limitación como una oportunidad de mejora para futuras evaluaciones. En conclusión, Random Forest presentó mejores resultados que Regresión Logística en las métricas analizadas, por lo que se considera la alternativa con mayor desempeño observado para este conjunto de datos. No obstante, sería conveniente realizar evaluaciones adicionales para comprobar la estabilidad de esta ventaja.

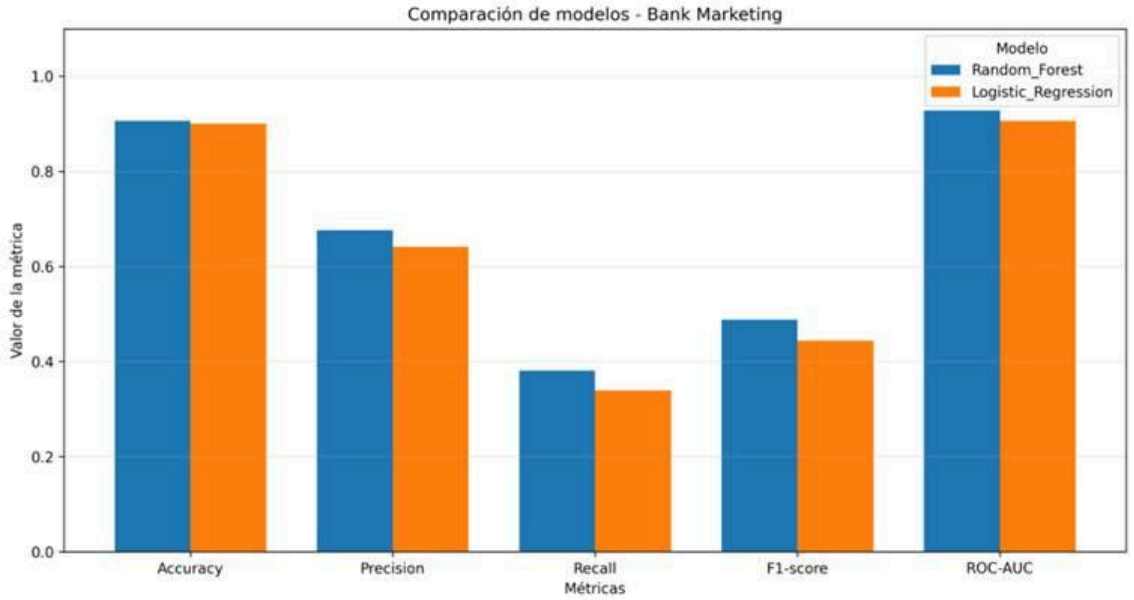


Figura 3. Comparación de métricas de prueba exportadas de MLflow. Random Forest supera a Regresión Logística en los indicadores mostrados, aunque ambos tienen recall inferior a 0.40.

9. Evidencia de uso de ML Flow

Para llevar un seguimiento organizado de los experimentos realizados, se utilizó MLflow, una herramienta que permite registrar y consultar los resultados obtenidos durante el desarrollo de modelos de aprendizaje automático. Inicialmente, los modelos de Regresión Logística y Random Forest fueron entrenados en Google Colab, donde se realizó la búsqueda de hiperparámetros y se calcularon las métricas necesarias para evaluar su desempeño.

Posteriormente, se instaló MLflow en una computadora local con el propósito de visualizar y conservar los resultados de manera organizada. Para ello, se creó el experimento denominado Bank_Marketing_Classification_Importado, en el cual se registraron las dos ejecuciones correspondientes a los algoritmos utilizados.

Dentro de MLflow se incorporaron las métricas de evaluación, incluyendo Accuracy, Precision, Recall, F1-score y ROC-AUC. Asimismo, se registraron los hiperparámetros seleccionados para cada modelo, lo que permitió consultar las configuraciones utilizadas durante el entrenamiento original.

También se añadieron como artefactos el archivo resultados_mlflow.csv, que contiene las métricas obtenidas, y la gráfica comparacion_modelos.png, que permite visualizar las diferencias de desempeño entre los algoritmos. Para comprobar el funcionamiento de la herramienta, se revisaron las ejecuciones desde la interfaz de MLflow y se recopilaron capturas de pantalla donde se muestran los experimentos, sus parámetros, métricas y artefactos.

Es importante señalar que los resultados fueron obtenidos durante el entrenamiento original en Google Colab y posteriormente importados a MLflow local, por lo que estas ejecuciones corresponden al registro de resultados existentes y no a nuevos entrenamientos. El uso de MLflow permitió organizar la información de los experimentos, facilitar la consulta de los resultados y fortalecer la trazabilidad del proyecto. Esta herramienta complementa el versionamiento realizado en GitHub y contribuye a incorporar prácticas relacionadas con MLOps.

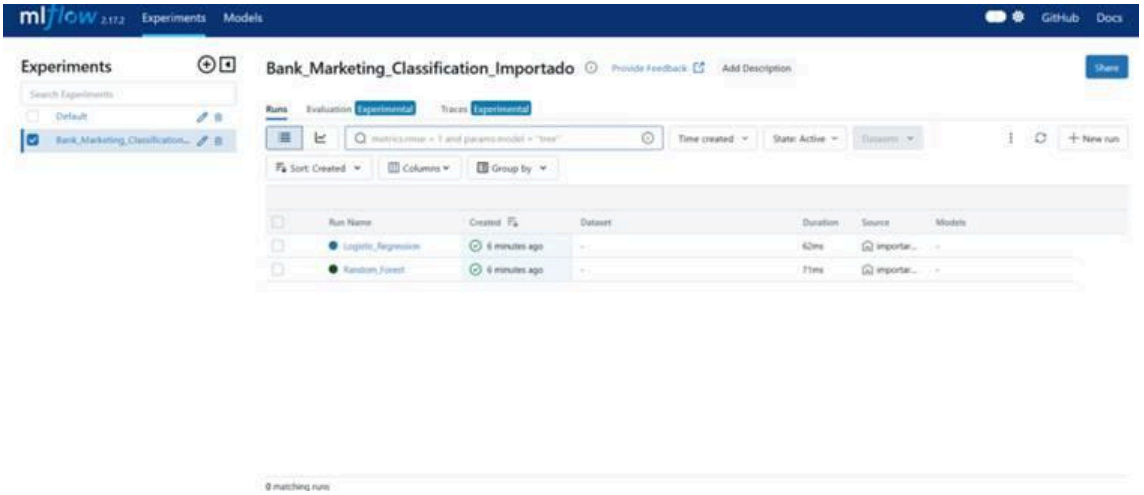


Figura 4. Experimento y ejecuciones importadas.



Figura 5. Parámetros y métricas de Regresión Logística.



Figura 6. Parámetros y métricas de Random Forest.

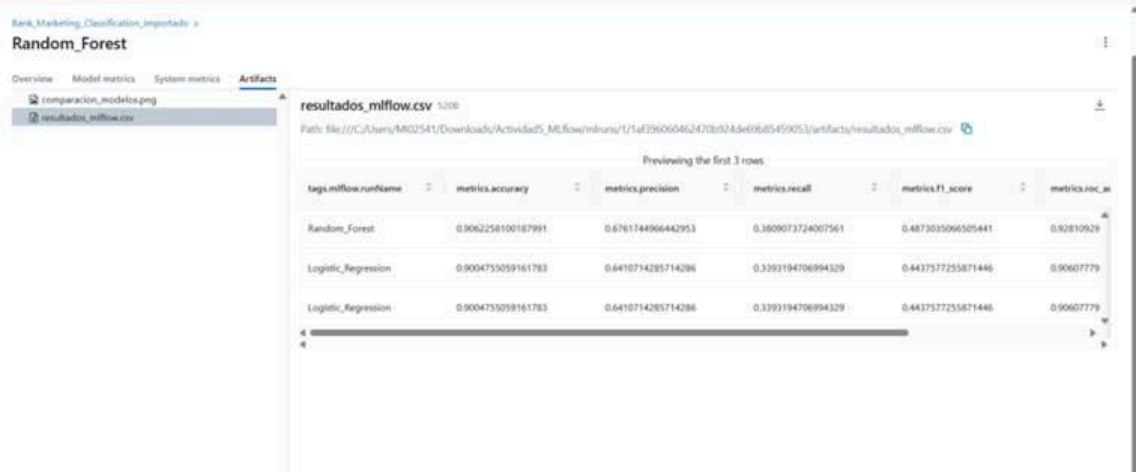


Figura 7. Vista de artefactos CSV en Random Forest.

10. Reflexión de deuda técnica y MLOps

Durante el desarrollo de la actividad se identificaron diferentes aspectos que permitieron comprender la importancia de mantener una adecuada organización y documentación en los procesos de aprendizaje automático. Uno de los principales retos fue la configuración de MLflow, ya que inicialmente se utilizó Google Colab para realizar el entrenamiento y seguimiento de los modelos. Sin embargo, se presentaron dificultades para visualizar los experimentos desde su interfaz, por lo que fue necesario instalar MLflow en un entorno local e importar posteriormente los resultados obtenidos.

Esta experiencia permitió identificar una oportunidad de mejora relacionada con la configuración del entorno de trabajo. En futuras actividades de este tipo sería conveniente establecer desde el inicio un sistema de almacenamiento persistente para los experimentos, evitando depender de entornos temporales y facilitando la recuperación de la información. Otro aspecto identificado fue la necesidad de fortalecer el tratamiento de valores faltantes y conservar resultados más detallados de la validación cruzada. Esto permitiría mejorar la calidad de los datos y realizar comparaciones estadísticas más completas entre los modelos.

En relación con MLOps, el proyecto permitió aplicar algunas prácticas importantes, como el versionamiento del código y los datos mediante GitHub, la documentación de los cambios realizados y el registro de parámetros, métricas y artefactos mediante MLflow. Estas herramientas facilitaron la organización del trabajo y permitieron mantener un seguimiento de los resultados obtenidos durante el desarrollo de los modelos. Como oportunidades de mejora, se considera importante automatizar algunas etapas del proceso, documentar las dependencias utilizadas y conservar los modelos entrenados para facilitar su reproducción y posterior utilización.

En conclusión, la actividad permitió comprender que el desarrollo de modelos de aprendizaje automático no consiste únicamente en obtener buenas métricas, sino también en garantizar que los procedimientos sean organizados,

documentados y reproducibles. La aplicación de prácticas de MLOps contribuye a mejorar la calidad, trazabilidad y mantenimiento de este tipo de proyectos.